

Audio Context Layer for Audio Question Answering

Technical Report — Proof of Concept Submission

1. Problem Statement

This project builds a system that listens to an audio recording and answers natural language questions about what happened in it. The questions go beyond identifying a single sound: the system is expected to reason about how many times something occurred, when events happened relative to each other, how long a sound lasted, and what kind of environment the audio suggests.

Rather than mapping raw audio directly to an answer, the system introduces an intermediate audio context layer. Audio is first converted into a structured, timestamped description of the events it contains, and a language model then answers questions by reasoning over that description rather than the waveform itself.

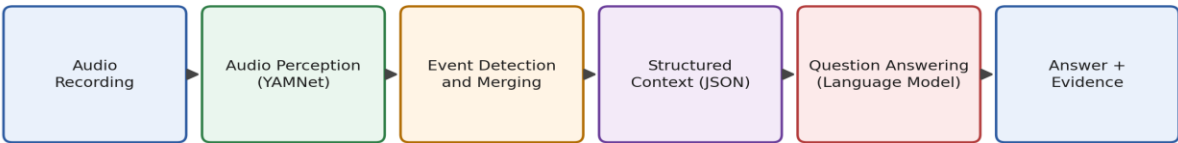


Figure 1. The overall pipeline: audio is converted to structured context before any question is answered.

A typical event in the context looks like this:

```
{
  "event_id": "event_001",
  "label": "dog",
  "start": 4.80,
  "end": 5.76,
  "duration": 0.96,
  "confidence": 0.82
}
```

Making this intermediate state explicit was the central design choice of the project. If a final answer turns out to be wrong, it becomes possible to check whether the sound was missed during perception, or whether the reasoning stage made a mistake despite being given correct information. This distinction turned out to matter a great deal for the error analysis in Section 7.

Question Types Supported

Type	Example
Apparent	What distinct sounds are present in the scene?
Counting	How many times does a dog bark?
Temporal	Which sound happened first?
Duration	How long did the siren last?

Type	Example
Comparison	Which of the two sounds lasted longer?
Yes / No	Was there a keyboard typing sound?
Causal reasoning	What kind of environment does the audio suggest?

In effect, the project treats audio question answering as two related problems: reconstructing a useful representation of what happened in the audio, and reasoning correctly over that representation once it exists.

2. Research Study

Before building the system, I looked at existing work in audio question answering and audio to language modeling to ground the design in established ideas rather than starting from nothing.

ClothoAQA

ClothoAQA treats audio question answering as more than sound classification, covering several distinct question types that each require a different kind of understanding of the clip. This motivated the decision to support multiple question types in this project rather than only checking whether individual sounds were detected.

AudioCaps

AudioCaps focuses on generating natural language captions from audio and demonstrates the value of converting acoustic information into language level information. Rather than generating a free form caption, this project stores a more structured intermediate representation, sound labels paired with precise timing, so that counting and temporal questions can be answered explicitly instead of being inferred from a description.

Audio Language Models (Pengi and related work)

Work on audio language models such as Pengi shows that a language model can reason effectively about information extracted from audio. The approach taken here differs in one respect: the language model never receives the raw waveform. It only receives an inspectable JSON representation produced by a separate perception stage, which keeps the reasoning process auditable.

What This Shaped

- Audio question answering should support several distinct question types, not just sound identification.
- Temporal information is essential for anything involving ordering or duration.
- Separating perception from reasoning makes it far easier to trace where a system actually fails.

The structured audio context layer sitting between perception and question answering is the main contribution of this prototype, and each of the three points above is reflected directly in its design.

3. Dataset

3.1 Base Audio Source

The project uses ESC 50 as its source of raw audio. ESC 50 contains 2,000 clips across 50 environmental sound classes, with 40 clips per class and each clip roughly five seconds long. Because these clips are short and isolated, they were combined into longer synthetic scenes rather than used individually. Twenty of the fifty available classes were selected for their usefulness in constructing coherent environments: car horn, chirping birds, clock tick, cow, crow, dog, door knock, engine, footsteps, hen, keyboard typing, laughing, mouse click, rain, rooster, sheep, siren, thunderstorm, water drops, and wind.

3.2 Synthetic Scene Construction

Each scene is built by placing several ESC 50 clips onto a timeline at controlled positions, producing longer recordings with known event boundaries, deliberate overlap, and at least one repeated sound per scene so that counting questions have a genuine answer. Five broad scene types were used: outdoor residential, rainy or stormy, indoor room, traffic or roadside, and farm or rural. An outdoor residential scene, for example, combines birds and wind in the background with footsteps, a dog, a car horn, or a crow in the foreground. The goal was not to reproduce real recordings exactly, but to build controlled scenes with exactly known ground truth.

Property	Value
Total scenes	150
Training scenes	100
Validation scenes	25
Test scenes	25
Scene duration	20 to 30 seconds (average 25.4s)
Events per scene	5 to 7 (average 5.86)
Scenes containing overlapping events	150 of 150

3.3 Dataset Splitting and Validation

Scenes were split using ESC 50's predefined folds rather than by random assignment, so that the same underlying recording could never appear in both a training scene and a test scene. Folds 1 through 3 were used for training, fold 4 for validation, and fold 5 for testing, giving 100, 25, and 25 scenes respectively. A dataset sanity check confirmed zero missing audio files, zero invalid timestamps, zero source recordings shared across splits, and overlapping events present in all 150 scenes.

3.4 Question Answering Pairs

Each scene was paired with roughly eight questions, giving 1,200 question and answer pairs in total, split 800 for training, 200 for validation, and 200 for testing.

Question Type	Count
Apparent	150
Causal reasoning	150
Comparison	150
Counting	150

Question Type	Count
Duration	150
Temporal	300
Yes / No	150
Total	1,200

Importantly, these question and answer pairs were never used to build the audio context at inference time. The system first processes the raw audio into a context representation, entirely independent of any question, and only then is a question asked against that already generated context. This keeps the ground truth answer completely separate from the perception process it is meant to evaluate.

4. Method and Design Decisions

4.1 Overall Architecture

The system has four working stages: audio is passed to a perception model, the model's frame level predictions above a confidence threshold are grouped into discrete events, the events are written into a timestamped JSON context, and a language model answers the question by reasoning over that context alone, returning an answer together with a confidence score and the specific event IDs it relied on. Keeping perception and reasoning as separate, independently replaceable components was intentional from the start, and it is what later made a precise error analysis possible.

4.2 Choosing the Perception Model

Four candidates were evaluated under an identical protocol: the same 25 test scenes, the same 248 five second evaluation windows, and the same 20 target classes. Three were pretrained AudioSet models used without further training (AST, PANNs CNN14, and YAMNet), and the fourth was a small convolutional network trained from scratch on this project's own scenes.

Model	Micro Precision	Micro Recall	Micro F1	Macro F1
AST	0.8089	0.2835	0.4198	0.3773
PANNs CNN14	0.7839	0.3482	0.4822	0.4311
YAMNet	0.6867	0.4844	0.5681	0.5214
Custom CNN	—	—	0.3811	0.3708

The custom network was trained mainly to confirm, rather than assume, that a small model trained on roughly a thousand windows could not match models pretrained on AudioSet's far larger corpus. It did not: its F1 trailed all three pretrained options. Among the pretrained models, AST and PANNs reached higher precision than YAMNet, but at substantially lower recall. Recall mattered more for this system: a false positive in the context can potentially be filtered later by the reasoning stage, but an event the perception model never detects is permanently unavailable to every stage downstream. YAMNet's higher recall and best overall F1 made it the clear choice, and this conclusion is specific to the measured performance in this experiment rather than a general claim about YAMNet's quality.

4.3 From Coarse Windows to Frame Level Context

An early version of the context layer aggregated YAMNet's predictions into five second windows, which proved too coarse: short events could disappear entirely under the averaging, and two separate occurrences of the same sound could be merged into one long event if they fell in overlapping windows. Switching to YAMNet's native frame level output, roughly 0.96 second frames spaced about 0.48 seconds apart, preserved far more temporal detail and was particularly important for repeated sounds, short events, and any question involving ordering or duration.

4.4 Event Detection and Merging

Frames are marked active when a class's confidence passes a threshold, and consecutive active frames of the same label are merged into one event provided the gap between them is not too large. Any resulting event shorter than a minimum duration is discarded as noise.

Parameter	Value
Confidence threshold	0.30
Minimum event duration	0.40 seconds
Maximum merge gap	1.50 seconds

The merge gap started at 0.75 seconds, which turned out to fragment continuous sounds whenever a model's frame level confidence dipped briefly between adjacent frames, an idling engine being a typical example. Widening the gap to 1.5 seconds, after inspecting several generated contexts by hand, reduced this fragmentation while still keeping genuinely separate repeated events apart, such as two footstep events with a clear pause between them. This is a real trade off rather than a free improvement: a wider gap that is pushed too far would eventually start merging events that are genuinely distinct. Section 6.3 reports the measured effect of this specific change on end to end accuracy.

4.5 Context Representation

Every detected event is stored with its label, start and end time, duration, confidence, and the underlying model frames that produced it, alongside metadata about the source audio and the perception model used. This makes the intermediate state genuinely inspectable: if a counting question is answered incorrectly, the individual detected events can be checked directly rather than only the final answer. The representation is also model independent, since any perception model that can emit the same event structure could replace YAMNet without changing anything downstream.

4.6 Question Answering

The reasoning stage uses GPT OSS 20B, accessed through Groq, and receives only the generated context and the question text, never the audio itself. It is instructed to answer strictly from the supplied context and to state plainly when the evidence is insufficient, rather than guessing. Every response is required to include the specific event IDs used as evidence, which keeps answers auditable and discourages the model from relying on outside world knowledge instead of what was actually detected.

4.7 Evaluation Approach

Different question types call for different scoring methods: apparent sound questions are scored with precision, recall, and F1 over the set of labels returned; counting, duration, and comparison questions are checked against their numeric or relational ground truth; temporal questions are checked against the known event ordering; and

causal reasoning questions are scored on whether the inferred environment matches the scene's intended environment, since these questions depend on interpreting several events together rather than matching one label.

The final evaluation issues one API request per question with strict validation of the response format, rather than batching several questions into a single request. This choice followed a concrete finding during development: batched requests occasionally returned answers in a different order than they were asked, and a positional fallback in the original scoring code would then silently attach an answer to the wrong question, producing evaluation numbers that did not reflect the system's real behaviour. Moving to one question per request removed this failure mode entirely.

4.8 Error Attribution Method

Every incorrect prediction was cross referenced against both the scene's ground truth manifest and its generated context to determine where the failure most likely originated. A perception or context error means the relevant event was missing, mislabelled, miscounted, or had a timestamp that differed meaningfully from the ground truth. A QA reasoning error means the relevant event was present and reasonably accurate in the context, yet the model still answered incorrectly. A mixed or ambiguous case is one where the context differs from ground truth for a relevant sound, but not clearly enough to say that difference caused the wrong answer. The purpose of this exercise is practical: it indicates whether future effort is better spent improving the perception model or the reasoning stage.

5. Experimental Setup

All four perception models were benchmarked on the same 25 test scenes, evaluated across 248 five second windows spanning the 20 target classes. The custom CNN was trained with the following configuration.

Setting	Value
Audio sample rate	16 kHz
Window / hop length	5.0 s / 2.5 s
Mel bands	64
FFT size / hop length	1024 / 512
Batch size	16
Optimizer	Adam, learning rate 0.001
Loss function	BCE with logits, class weighted
Epochs / random seed	15 / 42
Checkpoint selection	Best validation micro F1

Positive class weights were used in the loss function because the twenty target classes are not evenly represented across the dataset. One point worth stating plainly: the perception benchmark above used five second window aggregation for a fair, uniform comparison across all four models, while the final context pipeline used YAMNet's native frame level output. These are two different temporal resolutions applied to the same underlying model, and the benchmark numbers should be read as a model comparison rather than an exact description of the deployed context layer's operating point.

6. Results

6.1 Audio Perception

YAMNet produced the strongest measured F1 among the four evaluated approaches and also the highest recall, at 0.4844. The main trade off was precision: both AST and PANNs were more conservative and therefore more precise, while YAMNet recovered a larger share of the actual events present, which was the more useful property for this system.

Model	Micro F1	Macro F1
AST	0.4198	0.3773
PANNs CNN14	0.4822	0.4311
YAMNet	0.5681	0.5214
Custom CNN	0.3811	0.3708

6.2 Question Answering Results

The final evaluation on all 200 test questions, using the corrected single question evaluation pipeline described in Section 4.7, produced the following results.

Question Type	Result
Apparent (F1)	0.6621
Causal reasoning (environment accuracy)	0.9600
Comparison (accuracy)	0.0800
Counting (accuracy)	0.3200
Duration (accuracy)	0.2000
Temporal (accuracy)	0.5000
Yes / No (accuracy)	0.7600
Overall objective accuracy	0.3543 (62 / 175)

The overall objective figure is computed across the 175 questions outside the causal reasoning category, which is scored separately since it depends on interpreting a combination of events rather than matching a single label. Apparent and yes or no questions performed comparatively well whenever the relevant events were represented in the context. Counting, duration, comparison, and temporal questions were noticeably more sensitive to small errors in event detection and timing, a pattern examined in detail in Section 7.

6.3 Effect of the Merge Gap Change

The context construction change described in Section 4.4, widening the maximum merge gap from 0.75 to 1.5 seconds, was evaluated end to end on the same 200 test questions.

Metric	Gap = 0.75s	Gap = 1.5s
Apparent F1	66.21%	66.21%

Metric	Gap = 0.75s	Gap = 1.5s
Causal reasoning (environment)	80.0%	96.0%
Comparison	8.0%	8.0%
Counting	36.0%	32.0%
Duration	24.0%	20.0%
Temporal	46.0%	50.0%
Yes / No	80.0%	76.0%
Overall objective	36.0%	35.43%

The change produced a mix of small gains and small losses rather than a uniform improvement, and the net effect on overall objective accuracy is not meaningful given a test set of 175 objective questions. The clearest gain was in causal reasoning, plausibly because a less fragmented event set gives a more complete picture for inferring environment. Qualitative inspection of individual scenes did confirm that the wider gap reduced fragmentation of continuous sounds without merging events that should stay separate, which was the intended effect, even though it is not the dominant factor in the final accuracy numbers.

7. Error Analysis

Of the 200 scored test questions, 114 were answered incorrectly. Each of these was attributed to one of three categories using the method described in Section 4.8.

Category	Count	Percentage
Perception or context error	98	86.0%
Mixed or ambiguous	13	11.4%
QA reasoning error	3	2.6%
Total	114	100%

This attribution is based on an automated, rule based comparison against ground truth rather than manual review of every case, so individual labels should be read as informative rather than exact. The overall pattern is nonetheless clear: the large majority of incorrect answers trace back to the audio context being wrong for the specific sound the question concerned, not to the language model failing to reason over context that was actually correct.

Representative Examples

Missing event. One scene's ground truth included footsteps, rain, thunderstorm, water drops, and wind, but the generated context omitted footsteps entirely. A temporal question involving footsteps could not be answered correctly as a result, even though the model's reasoning over the context it was given was sound.

Undercounted event. In another scene, the manifest contained two separate water drop events, but the context contained only one, since the second occurrence had been merged into the first. The counting answer was wrong even though the model correctly counted what it was shown.

Truncated duration. A siren lasting close to five seconds in the manifest was represented in the context as lasting only 3.36 seconds. This is a known limitation of frame level thresholding on sustained sounds whose

confidence does not stay uniformly high across their full length, and it directly affected every duration question involving that event.

Genuine reasoning error. A small number of cases had accurate context but a wrong answer regardless. One yes or no question had enough evidence in the context to support a confident answer, but the model responded that it could not determine one. These cases were classified as QA reasoning errors and, while too few to generalize from, point toward two narrow behaviours worth addressing in a future iteration: reluctance to assert a confident negative, and occasional mistakes in temporal ordering.

Taken together, this analysis indicates that the dominant bottleneck in the current prototype is audio perception and context reconstruction, not language reasoning, which is exactly the distinction the architecture was designed to make visible.

8. Loss Curves

The custom CNN was trained for 15 epochs using class weighted binary cross entropy loss. Training loss decreased steadily throughout, while validation loss was noisier and began to drift upward after around epoch 11, which is why the checkpoint at epoch 11 (validation micro F1 of 0.3848) was selected rather than the final epoch.

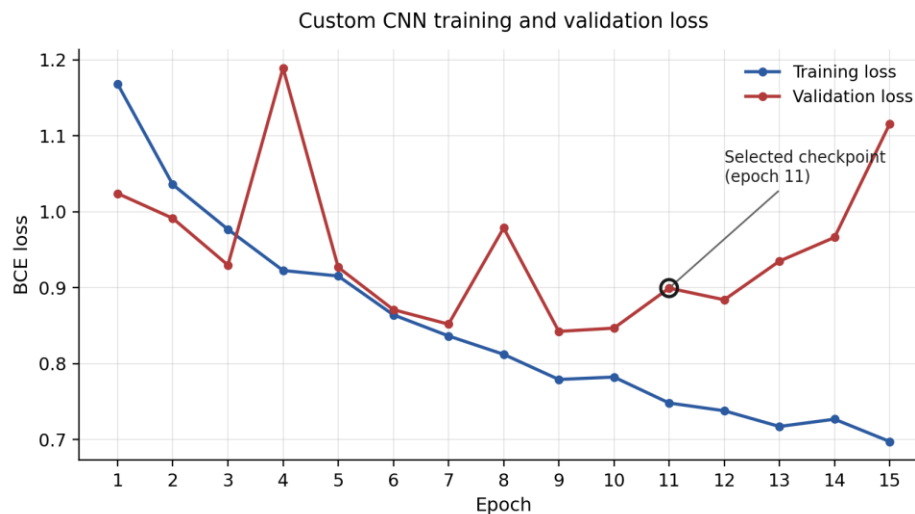


Figure 2. Training and validation loss for the custom CNN across 15 epochs.

The custom model remains a useful supervised baseline and a demonstration of a complete, reproducible training pipeline, but its final test F1 (0.3811) fell short of the pretrained YAMNet model. Given the limited training data and the short development window available for this project, a pretrained model was ultimately the stronger foundation for the context layer.

9. Observations and Limitations

Main Observations

The single clearest finding across this project is that the quality of the context representation drives the final answer quality almost entirely. When an event is missing from the context, the language model has no way to

recover it, and inaccurate timestamps propagate directly into duration, ordering, and comparison questions. Because the context is inspectable rather than hidden inside an end to end model, these failure modes could be identified and quantified rather than merely suspected. A related, smaller finding is that frame level predictions were substantially more useful for temporal reasoning than coarse aggregated windows, even though they came with a fragmentation problem of their own that had to be addressed separately.

Limitations

Synthetic audio. Scenes are built from ESC 50 clips rather than recorded as continuous real world audio, so they lack the background noise, reverberation, and natural sound interaction of genuine recordings.

Dataset scale. 150 scenes and 200 test questions are sufficient to demonstrate the approach but too small to support broad claims about audio question answering in general.

Perception without fine tuning. YAMNet was used exactly as pretrained. Some of the twenty target classes do not map cleanly onto YAMNet's original AudioSet vocabulary, which introduces classification errors that fine tuning could likely reduce.

Approximate timestamps. Event boundaries are reconstructed from model confidence rather than measured directly, so they are estimates. This limitation affects duration and temporal questions more than any other category, as shown in Section 7.

Benchmark and deployment mismatch. The perception benchmark in Section 6.1 used five second window aggregation for fair comparison across models, while the deployed context layer uses native frame level predictions. The benchmark should be read as a model selection experiment, not an exact description of deployed performance.

External QA model dependency. The reasoning stage depends on a third party language model accessed through an API. A different model, or a different version of the same model, could produce different results from an identical context.

10. Conclusion

This project implements a proof of concept audio question answering system built around an explicit, structured audio context layer rather than a single end to end model. Audio is first converted into timestamped events, and a language model then reasons over those events to answer natural language questions spanning seven distinct question types.

Four perception approaches were compared under an identical protocol, with YAMNet producing the strongest measured performance and being selected for the final pipeline. The context construction stage was iterated once, moving from coarse window aggregation to native frame level reconstruction, after this was found to fragment and merge events incorrectly. End to end evaluation on 200 test questions showed strong performance on apparent, causal, and yes or no questions, and weaker performance on counting, duration, and comparison questions.

The error analysis in Section 7 traced the large majority of these weaker results to the audio context itself rather than to the language model's reasoning, which is the central justification for separating perception from reasoning in the first place: it made this diagnosis possible. The most promising direction for improving this

prototype is therefore better event boundary reconstruction, and possibly fine tuning the perception model on this project's own scene distribution, rather than changes to the reasoning stage.